

AI ASSISTED CODING

ASSIGNMENT : 20.1

NAME: KADARLA HASINI

HT.NO : 2403A52062

BATCH NO : 01

Task 1 – Input Validation Check

Task:

Analyze an AI-generated **Python login script** for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

PROMPT:

Generate a simple Python program for username and password login. The program should ask the user for a username and password, and then check if they match a predefined set of credentials. Do not include any advanced security features or input validation

CODE AND OUTPUT:

```
[1]
✓ 52s ▶ def login():
    """Simulates a simple username and password login."""

    valid_username = "user123"
    valid_password = "password123"

    entered_username = input("Enter username: ")
    entered_password = input("Enter password: ")

    if entered_username == valid_username and entered_password == valid_password:
        print("Login successful!")
    else:
        print("Invalid username or password.")

    if __name__ == "__main__":
        login()

... Enter username: user123
Enter password: password123
Login successful!
```

CODE EXPLANATION:

This Python script simulates a basic login. It defines a `login()` function with hardcoded valid username and password. The function prompts the user for input using `input()`, then compares the entered credentials to the valid ones. If they match, it prints "Login successful!"; otherwise, it prints "Invalid username or password." The `if __name__ == "__main__":` block ensures the `login()` function runs when the script is executed directly. This example is intentionally simple and lacks security measures like input validation for demonstration purposes.

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to **SQL injection** (e.g., using string concatenation in queries).
- Refactor using **parameterized queries (prepared statements)**.

Expected Output:

- A secure database query script resistant to SQL injection.

PROMPT:

Generate a Python script that connects to an SQLite database, creates a simple 'users' table with 'id' and 'username' columns, inserts a few sample users, and then includes a function to fetch user details by username using a SQL query constructed with string concatenation. The script should demonstrate how a user input could be used in the query

CODE AND OUTPUT:

```
[1]
✓ Os

import sqlite3

def create_database():
    conn = sqlite3.connect('user_database.db')
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL
        )
    ''')

    cursor.execute("INSERT INTO users (username) VALUES (?)", ('alice',))
    cursor.execute("INSERT INTO users (username) VALUES (?)", ('bob',))
    cursor.execute("INSERT INTO users (username) VALUES (?)", ('charlie',))

    conn.commit()
    conn.close()

def get_user_details_vulnerable(username):
    conn = sqlite3.connect('user_database.db')
    cursor = conn.cursor()

    # Vulnerable query using string concatenation
    query = "SELECT * FROM users WHERE username = '" + username + "';"
    print(f"Executing query: {query}")

    try:
        cursor.execute(query)
        user = cursor.fetchone()
        return user
    except sqlite3.Error as e:
```

```
[1] 0s
except sqlite3.Error as e:
    print(f"Database error: {e}")
    return None
finally:
    conn.close()

# Create the database and insert initial data
create_database()

# Example of using the vulnerable function
print("Attempting to fetch user 'alice':")
user_details = get_user_details_vulnerable("alice")
print(f"Result: {user_details}")

print("\nAttempting to fetch user with a malicious input:")
malicious_input = "' OR '1'='1'"
user_details_malicious = get_user_details_vulnerable(malicious_input)
print(f"Result: {user_details_malicious}")
```

def get_user_details_vulnerable(username)
View source
<function get_user_details_vulnerable at 0x7827975747c0>

```
... Attempting to fetch user 'alice':  
Executing query: SELECT * FROM users WHERE username = 'alice';  
Result: (1, 'alice')  
  
Attempting to fetch user with a malicious input:  
Executing query: SELECT * FROM users WHERE username = '' OR '1'='1';  
Result: (1, 'alice')
```

CODE EXPLANATION:

The code includes a function `get_user_details_vulnerable` that fetches user details from an SQLite database. It constructs the SQL query using string concatenation: `query = "SELECT * FROM users WHERE username = '" + username + "'"`. This method is vulnerable to SQL injection because user input (`username`) is directly inserted into the query string. A malicious user can provide input that changes the query's logic, potentially accessing unauthorized data or manipulating the database. The example with the malicious input `' OR '1'='1'` demonstrates how this vulnerability can be exploited to bypass the intended username check.

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated **HTML form with JavaScript** for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.

- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

PROMPT:

Generate the HTML code for the secure feedback form we discussed, but this time include a Content Security Policy (CSP) header or meta tag that restricts where scripts can be loaded from. For example, allow scripts only from the same origin ('self')

CODE AND OUTPUT:

```
[5]
✓ Os
▶ # Example of saving to a secure HTML file:
with open("secure_feedback_form.html", "w") as f:
    f.write("""
<!DOCTYPE html>
<html>
<head>
<title>Feedback Form (Secure)</title>
</head>
<body>

<h1>Feedback Form (Secure)</h1>

<form id="feedbackFormSecure">
  <label for="feedbackSecure">Your Feedback:</label><br>
  <textarea id="feedbackSecure" name="feedbackSecure" rows="4" cols="50"></textarea><br><br>
  <button type="button" onclick="displayFeedbackSecure()">Submit Feedback</button>
</form>

<div id="outputSecure">
  <h2>Your Feedback:</h2>
</div>

<script>
function escapeHTML(str) {
  const div = document.createElement('div');
  div.appendChild(document.createTextNode(str));
  return div.innerHTML;
}

function displayFeedbackSecure() {
  const feedback = document.getElementById("feedbackSecure").value;
  const escapedFeedback = escapeHTML(feedback); // Sanitize input
  document.getElementById("outputSecure").innerHTML += "<p>" + escapedFeedback + "</p>";
}

```

```
[5]
✓ Os

function escapeHTML(str) {
    const div = document.createElement('div');
    div.appendChild(document.createTextNode(str));
    return div.innerHTML;
}

function displayFeedbackSecure() {
    const feedback = document.getElementById("feedbackSecure").value;
    const escapedFeedback = escapeHTML(feedback); // Sanitize input
    document.getElementById("outputSecure").innerHTML += "<p>" + escapedFeedback + "</p>";
}
</script>

</body>
</html>
""")

print("HTML code saved to secure_feedback_form.html")

HTML code saved to secure_feedback_form.html
```

CODE EXPLANATION:

This code creates a secure HTML feedback form. It includes a text area for user input and a button. When the button is clicked, JavaScript retrieves the feedback. Crucially, before displaying the feedback on the page, a function called `escapeHTML` is used to sanitize the input. This function converts potentially harmful characters like `<` and `>` into their HTML-safe equivalents, preventing any malicious scripts from being executed in the user's browser and mitigating XSS vulnerabilities.

Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Task:

Scenario:

Students pick an **AI-generated project snippet** (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest **secure coding practices** to fix issues.
- Compare insecure vs secure versions side by side.

Expected Output:

A security-audited code snippet with documented vulnerabilities and fixes.

PROMPT:

Act as a security expert and audit the following AI-generated code snippet for potential vulnerabilities. Identify and explain any security flaws you find. Then, suggest secure coding practices and provide a corrected, secure version of the code. Finally, present a side-by-side comparison of the original, insecure code and the corrected, secure code, highlighting the changes made to address the vulnerabilities

CODE AND OUTPUT:

```
[1]
✓ Os  # Replace this with your AI-generated code snippet
insecure_code = """
# Example insecure code snippet (e.g., a simple login form)
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "password123":
    print("Login successful!")
else:
    print("Invalid credentials.")
"""

print("Insecure code:")
print(insecure_code)

... Insecure code:

# Example insecure code snippet (e.g., a simple login form)
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "password123":
    print("Login successful!")
else:
    print("Invalid credentials.")
```

CODE EXPLANATION:

This code defines a multi-line string variable called `insecure_code` which contains a simple Python script for a login form. It then prints the label "Insecure code:" followed by the content of the `insecure_code` variable. The example code itself is very basic and demonstrates a hardcoded username and password check, which is highly insecure for any real-world application.